

Exp 10-Write the python program to implement A* algorithm.

Aim:

To write a python program to implement A* algorithm.

Algorithm:

Step1 :Initialize the open list with the starting node and the closed list as an empty set.

Step 2: Initialize the g score of the starting node to be 0 and the f score of the starting node to be the heuristic estimate of the distance to the goal.

Step 3: While the open list is not empty, select the node with the lowest f score and set it as the current node.

Step 4: If the current node is the goal node, return the reconstructed path.

Remove the current node from the open list and add it to the closed list.

Step 5: For each neighbor of the current node, calculate the tentative g score as the g score of the current node plus the distance between the current node and the neighbor.

Step 6: If the neighbor is already in the closed list, skip it.

Step 7: If the neighbor is not in the open list, add it and set its g score and f score.

Step 8 :If the neighbor is in the open list but the tentative g score is greater than or equal to its current g score, skip it.

Step 9 : Otherwise, update the came from dictionary, g score, and f score of the neighbor.

Step 10 : Repeat steps 3-10 until the open list is empty.

If the goal node was never reached, return failure.

Program:

```
import heapq

def astar(start, goal, neighbors_fn, heuristic_fn):
    """
    Find the shortest path from start to goal using A* Algorithm.
    """

    frontier = []
    heapq.heappush(frontier, (0, start))

    came_from = {}
    cost_so_far = {}

    came_from[start] = None
    cost_so_far[start] = 0

    while frontier:

        _, current = heapq.heappop(frontier)

        if current == goal:
            break

        for neighbor in neighbors_fn(current):
```

```
new_cost = cost_so_far[current] + 1
```

```
if (neighbor not in cost_so_far or  
    new_cost < cost_so_far[neighbor]):
```

```
    cost_so_far[neighbor] = new_cost
```

```
    priority = new_cost + heuristic_fn(neighbor, goal)
```

```
    heapq.heappush(frontier, (priority, neighbor))
```

```
    came_from[neighbor] = current
```

```
path = [goal]
```

```
current = goal
```

```
while current != start:
```

```
    current = came_from[current]
```

```
    path.append(current)
```

```
path.reverse()
```

```
return path, cost_so_far[goal]
```

Output:

```
# Print the result print("Shortest  
path:", path) print("Cost:", cost)
```

In this example, we have a 5x5 grid where 0 represents passable cells and 1 represents obstacles. The `grid_neighbors` function defines the neighbors of a node on the grid, considering only right, left, up, and down movements. The `grid_heuristic` function calculates the Manhattan distance between two nodes on the grid.

By calling the `astar` function with the start and goal nodes, along with the `grid_neighbors` and `grid_heuristic` functions, we can find the shortest path from the start (0, 0) to the goal (4, 4) on the grid. The resulting path and cost are then printed.

Result:

Thus the Program was executed and the output was found to be Correct.